

在 GKE 上使用 IIm-d 部署分離式 TPU vLLM 推論

來源：<https://codelabs.developers.google.com/next26/aiinfra-learning-pod/screen2-advanced-inferencing-part-2?hl=zh-tw>

步驟數：9

使用 IIm-d 和 GKE，在分離式 v6e TPU 上部署 Qwen3-32B 模型。

目錄

1. [1. 簡介](#)
2. [2. 事前準備](#)
3. [3. 建立自訂網路](#)
4. [4. 佈建 GKE 叢集](#)
5. [5. 建立預留 TPU 節點集區](#)
6. [6. 部署 llm-d 服務](#)
7. [7. 測試部署作業回應](#)
8. [8. 清除所用資源](#)
9. [9. 恭喜](#)

步驟 1 · 約 0 分鐘

1. 簡介

在本程式碼研究室中，您將瞭解如何使用 Google Cloud TPU，在 Google Kubernetes Engine (GKE) 上部署高效能的非聚合式推論服務。您將使用 **llm-d** (適用於分散式 LLM 服務的開放原始碼框架)，在多個 TPU 主機之間區隔預填和解碼階段、設定共用 KV 快取，以及 GKE Inference Gateway。

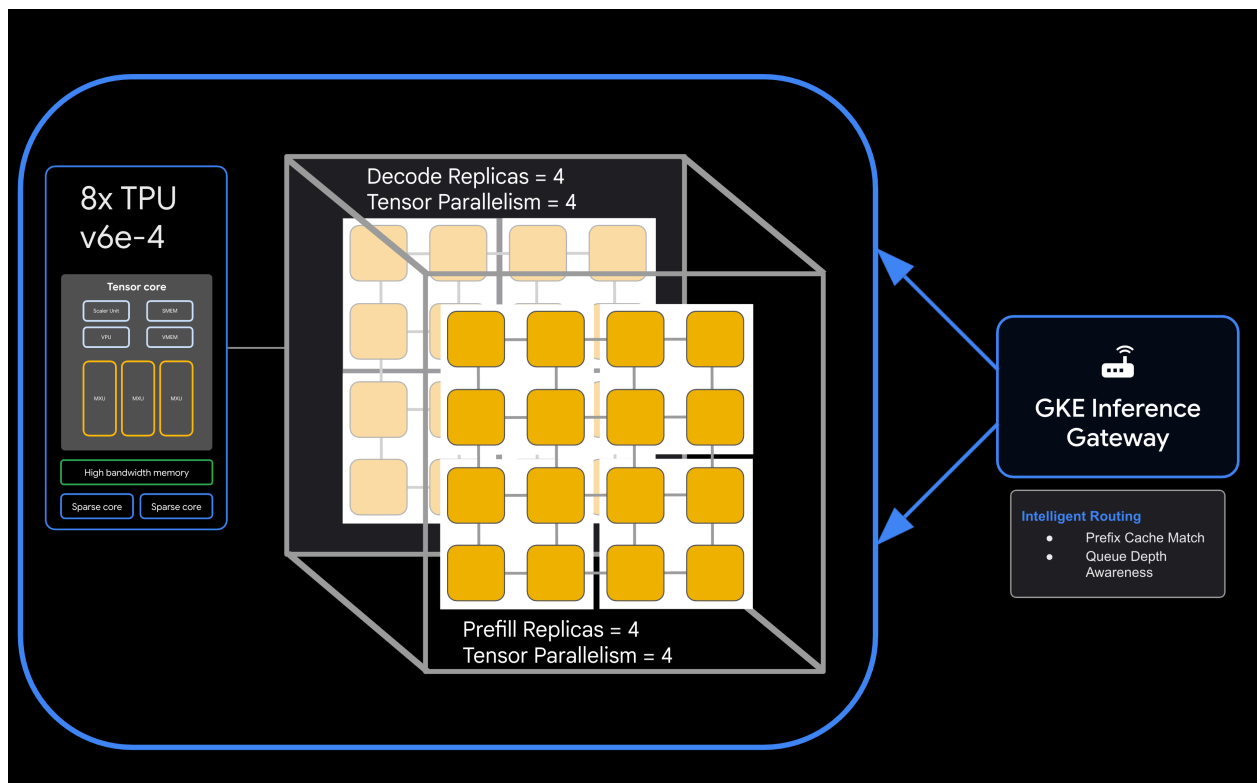
這個設定會模擬正式環境，以高處理量和低延遲提供 **Qwen3-32B** 等大型模型。

學習內容

- 建立自訂虛擬私有雲網路，並為加速器流量設定最佳 MTU。
- 佈建 GKE 叢集，並使用 GCS Fuse CSI 驅動程式和 Ray Operator 外掛程式。
- 為 TPU v6e 配量 (總共 32 個晶片) 建立 8 個專屬節點集區。
- 設定 Workload Identity 和權限，以便存取 GCS。
- 部署 **llm-d**，管理 Qwen3-32B 模型的非聚合式服務。
- 使用基準測試驗證部署作業。

架構

[llm-d disaggregated serving architecture showing model split into 4 2x2 replicas of prefill and the same for decode]



軟硬體需求

- 已啟用計費功能的 Google Cloud 專案。
- TPU v6e 資源的 **Google Cloud** 預訂 (32 個晶片，**ct6e-standard-4t**)。
- [Hugging Face 使用者存取權杖](#)，用於下載模型權重。
- Cloud Shell 或已安裝 **gcloud**、**kubectl** 和 **helm** 的本機終端機。

重要需求：本程式碼研究室需要透過特定預訂項目分配 **32 個 v6e (Trillium) 晶片**。這是專為 AI 基礎架構展示設計的實驗室，成本高昂。

- **預計時間：**60 分鐘
 - **預估費用：**本實驗室會用到大量 TPU 資源，完成專案至少需要 \$60 美元。請務必在完成練習後立即按照清除步驟操作。
-

步驟 2 · 約 5 分鐘

2. 事前準備

建立或選取 Google Cloud 專案

1. 在 [Google Cloud 控制台](#) 中，選取或建立 Google Cloud 專案。
2. 確認 Cloud 專案已啟用計費功能。

啟動 Cloud Shell

1. 點選 Google Cloud 控制台頂端的「啟用 Cloud Shell」。
2. 驗證：

□□□

```
gcloud auth list
```

3. 確認專案：

□□□

```
gcloud config get project
```

4. 視需要設定：

```
export PROJECT_ID=<YOUR_PROJECT_ID>
gcloud config set project $PROJECT_ID
```

啟用 API

啟用必要的 Google Cloud 服務：

```
gcloud services enable \
  container.googleapis.com \
  compute.googleapis.com \
  iam.googleapis.com \
  cloudresourcemanager.googleapis.com
```

設定環境變數

在殼層中定義下列變數。將 <YOUR_ZONE> 換成您分配到的 TPU 區域，<YOUR_RESERVATION_NAME> 換成預訂 ID，<YOUR_HUGGING_FACE_TOKEN> 換成您的權杖。

```
export PROJECT_ID=$(gcloud config get-value project)
export ZONE="<YOUR_ZONE>" # e.g., us-east5-a
export REGION=${ZONE%-*}
export NAMESPACE=default
export CLUSTER_NAME="qwen-serving-cluster"
export GVNIC_NETWORK_PREFIX="qwen-serving"
export RESERVATION_NAME="<YOUR_RESERVATION_NAME>"
export HF_TOKEN="<YOUR_HUGGING_FACE_TOKEN>"
```

3. 建立自訂網路

如要使用分離式服務，必須進行特定網路設定，才能處理預填和解碼節點之間的高頻寬流量。

1. 建立虛擬私有雲網路，並將 MTU 設為較大的值 (8896)，以提升加速器通訊效率：

```
gcloud compute --project=${PROJECT_ID} \
  networks create ${GVNIC_NETWORK_PREFIX}-main \
  --subnet-mode=auto \
  --bgp-routing-mode=regional \
  --mtu=8896
```

2. 建立叢集的子網路：

```
gcloud compute --project=${PROJECT_ID} \
  networks subnets create ${GVNIC_NETWORK_PREFIX}-tpu \
  --network=${GVNIC_NETWORK_PREFIX}-main \
  --region=${REGION} \
  --range=10.10.0.0/18
```

3. 建立僅限 **Proxy** 的子網路，這是 GKE Gateway API 的必要條件：

```
gcloud compute networks subnets create ${GVNIC_NETWORK_PREFIX}-proxy \
  --purpose=REGIONAL_MANAGED_PROXY \
  --role=ACTIVE \
  --region=${REGION} \
  --network=${GVNIC_NETWORK_PREFIX}-main \
  --range=172.16.0.0/26
```

4. 建立防火牆規則，允許內部通訊：

```
gcloud compute --project=${PROJECT_ID} firewall-rules create ${GVNIC_NETWORK_PREFIX}-
allow-internal \
  --network=${GVNIC_NETWORK_PREFIX}-main \
  --allow=all \
  --source-ranges=172.16.0.0/12,10.0.0.0/8 \
  --description="Allow all internal traffic within the network."
```

步驟 4 · 約 8 分鐘

4. 佈建 GKE 叢集

建立設定為支援 GCS Fuse 掛接和 Ray Operator 工作負載的 Standard GKE 叢集。

1. 建立叢集：

```
gcloud container clusters create ${CLUSTER_NAME} \
  --project=${PROJECT_ID} \
  --location=${REGION} \
  --release-channel=rapid \
  --machine-type=e2-standard-4 \
  --network=${GVNIC_NETWORK_PREFIX}-main \
  --subnetwork=${GVNIC_NETWORK_PREFIX}-tpu \
  --num-nodes=1 \
  --gateway-api=standard \
  --enable-managed-prometheus \
  --enable-dataplane-v2 \
  --enable-dataplane-v2-metrics \
  --workload-pool=${PROJECT_ID}.svc.id.goog \
  --
addons=HttpLoadBalancing,GcsFuseCsiDriver,RayOperator,HorizontalPodAutoscaling,NodeLocalDNS \
  --enable-ip-alias
```

2. 擷取叢集憑證：

```
gcloud container clusters get-credentials ${CLUSTER_NAME} --region=${REGION}
```

3. 建立 Hugging Face Secret：

```
kubectl create secret generic llm-d-hf-token \
  --from-literal=hf_api_token=${HF_TOKEN} \
  --dry-run=client -o yaml | kubectl apply -f -
```

步驟 5 · 約 10 分鐘

5. 建立預留 TPU 節點集區

使用預留項目，為 TPU v6e 配量佈建 8 個專屬節點集區。

執行下列迴圈，建立 8 個節點集區：

```
for i in {1..8}
do
  gcloud beta container node-pools create "tpu-v6e-single-$i" \
    --project=${PROJECT_ID} \
    --cluster=${CLUSTER_NAME} \
    --region=${REGION} \
    --node-locations=${ZONE} \
    --machine-type=ct6e-standard-4t \
    --tpu-topology=2x2 \
    --num-nodes=1 \
    --reservation-affinity=specific \
    --reservation=${RESERVATION_NAME} \
    --workload-metadata=GKE_METADATA &
done
```

等待所有節點建立完成並加入叢集。你可以透過 `kubectl get nodes` 查看狀態。

步驟 6 · 約 10 分鐘

6. 部署 llm-d 服務

現在您要部署 `llm-d` 架構，管理解除匯總的放送作業。

1. 安裝 **Helm**，部署 llm-d 圖表：

```
curl -fsSL -o get_helm.sh https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-4
chmod 700 get_helm.sh
./get_helm.sh
```

2. 複製 **llm-d** 並安裝必要依附元件：

```
git clone https://github.com/llm-d/llm-d.git
# When using yq alongside Helm, you almost always want the version by Mike Farah
(mikefarah/yq). We remove the most common yq installation before reinstalling
sudo rm -rf /usr/local/bin/yq
cd llm-d
./helpers/client-setup/install-deps.sh
```

3. 準備自訂 `values_tpu.yaml`，為叢集設定分離式服務：

```
cat <<EOF > llm-d/guides/pd-disaggregation/ms-pd/values_tpu.yaml
multinode: false

# Configure accelerator type for Google TPU
accelerator:
  type: google

modelArtifacts:
  uri: "hf://Qwen/Qwen3-32B"
  size: 200Gi
  authSecretName: "llm-d-hf-token"
  name: "Qwen/Qwen3-32B"
  labels:
    llm-d.ai/inference-serving: "true"
    llm-d.ai/guide: "pd-disaggregation"
    llm-d.ai/hardware-variant: "tpu"
    llm-d.ai/hardware-vendor: "google"
    llm-d.ai/model: "Qwen3-32B"

tracing:
  enabled: true
  otlpEndpoint: "localhost:4317"
  serviceNames:
    routingProxy: "routing-proxy"
  sampling:
    sampler: "always_off"
    samplerArg: "0"

routing:
  servicePort: 8000
  proxy:
    image: ghcr.io/llm-d/llm-d-routing-sidecar:v0.5.0
    connector: nixlv2
    secure: false

decode:
  parallelism:
    tensor: 4
  create: true
  replicas: 4
  modelCommand: custom
  extraConfig:
    nodeSelector:
      cloud.google.com/gke-tpu-accelerator: "tpu-v6e-slice"
      cloud.google.com/gke-tpu-topology: "2x2"
  monitoring:
    podmonitor:
      enabled: true
      portName: "vllm"
      path: "/metrics"
      interval: "30s"
  containers:
```

```

- name: "vllm"
image: "vllm/vllm-tpu:nightly"
command:
  - "/bin/bash"
  - "-c"
  - |
    # ROLE: kv_consumer (Receives KV cache from prefill)
    KV_CONFIG="{\"kv_connector\": \"TPUConnector\", \"kv_connector_module_path\" :
    \"tpu_inference.distributed.tpu_connector\", \"kv_role\": \"kv_consumer\", \"kv_ip\" :
    \"${POD_IP}\"}"
    echo "KV_CONFIG=${KV_CONFIG}"
    python3 -m vllm.entrypoints.openai.api_server \
    --model "Qwen/Qwen3-32B" \
    --port 8200 \
    --tensor-parallel-size 4 \
    --kv-transfer-config "${KV_CONFIG}" \
    --disable-uvicorn-access-log \
    --max-num-seqs 256 \
    --block-size 128 \
    --gpu-memory-utilization 0.90 \
    --max-model-len 8192

env:
  - name: POD_IP
  valueFrom:
    fieldRef:
      fieldPath: status.podIP
  - name: TPU_SIDE_CHANNEL_PORT
  value: "9600"
  - name: TPU_KV_TRANSFER_PORT
  value: "9100"

ports:
  - containerPort: 8200
    name: vllm
    protocol: TCP
  - containerPort: 9100
    name: tpu-kv-transfer
    protocol: TCP
  - containerPort: 9600
    name: tpu-coord
    protocol: TCP

resources:
  limits:
    memory: 64Gi
    cpu: "16"
    google.com/tpu: 4
  requests:
    memory: 64Gi
    cpu: "16"
    google.com/tpu: 4
mountModelVolume: true
volumeMounts:
  - name: metrics-volume

```

```
    mountPath: /.config
    - name: shm
    mountPath: /dev/shm
    - name: torch-compile-cache
    mountPath: /.cache
  startupProbe:
    httpGet:
      path: /health
      port: vllm
    initialDelaySeconds: 15
    periodSeconds: 30
    timeoutSeconds: 5
    failureThreshold: 120
  livenessProbe:
    httpGet:
      path: /health
      port: vllm
    periodSeconds: 10
    timeoutSeconds: 5
    failureThreshold: 3
  readinessProbe:
    httpGet:
      path: /v1/models
      port: vllm
    periodSeconds: 5
    timeoutSeconds: 2
    failureThreshold: 3
volumes:
  - name: metrics-volume
    emptyDir: {}
  - name: shm
    emptyDir:
      medium: Memory
      sizeLimit: "16Gi"
  - name: torch-compile-cache
    emptyDir: {}

prefill:
parallelism:
  tensor: 4
create: true
replicas: 4
modelCommand: custom
extraConfig:
  nodeSelector:
    cloud.google.com/gke-tpu-accelerator: "tpu-v6e-slice"
    cloud.google.com/gke-tpu-topology: "2x2"
monitoring:
  podmonitor:
    enabled: true
    portName: "vllm"
    path: "/metrics"
```

```

    interval: "30s"
containers:
  - name: "vllm"
    image: "vllm/vllm-tpu:nightly"
    command:
      - "/bin/bash"
      - "-c"
      - |
        # ROLE: kv_producer (Sends KV cache to decode)
        KV_CONFIG="{\"kv_connector\": \"TPUConnector\", \"kv_connector_module_path\" :
        \"tpu_inference.distributed.tpu_connector\", \"kv_role\": \"kv_producer\", \"kv_ip\" :
        \"${POD_IP}\"}"

        echo "KV_CONFIG=${KV_CONFIG}"
        python3 -m vllm.entrypoints.openai.api_server \
        --model "Qwen/Qwen3-32B" \
        --port 8200 \
        --tensor-parallel-size 4 \
        --kv-transfer-config "${KV_CONFIG}" \
        --disable-uvicorn-access-log \
        --enable-chunked-prefill \
        --block-size 128 \
        --gpu-memory-utilization 0.90 \
        --max-model-len 8192

    env:
      - name: POD_IP
        valueFrom:
          fieldRef:
            fieldPath: status.podIP
      - name: TPU_SIDE_CHANNEL_PORT
        value: "9600"
      - name: TPU_KV_TRANSFER_PORT
        value: "9100"
    ports:
      - containerPort: 8200
        name: vllm
        protocol: TCP
      - containerPort: 9100
        name: tpu-kv-transfer
        protocol: TCP
      - containerPort: 9600
        name: tpu-coord
        protocol: TCP
    resources:
      limits:
        memory: 64Gi
        cpu: "16"
        google.com/tpu: 4
      requests:
        memory: 64Gi
        cpu: "16"
        google.com/tpu: 4
    mountModelVolume: true

```

```

volumeMounts:
  - name: metrics-volume
    mountPath: /.config
  - name: shm
    mountPath: /dev/shm
  - name: torch-compile-cache
    mountPath: /.cache
startupProbe:
  httpGet:
    path: /health
    port: vllm
    initialDelaySeconds: 15
    periodSeconds: 30
    timeoutSeconds: 5
    failureThreshold: 120
livenessProbe:
  httpGet:
    path: /health
    port: vllm
    periodSeconds: 10
    timeoutSeconds: 5
    failureThreshold: 3
readinessProbe:
  httpGet:
    path: /v1/models
    port: vllm
    periodSeconds: 5
    timeoutSeconds: 2
    failureThreshold: 3
volumes:
  - name: metrics-volume
    emptyDir: {}
  - name: shm
    emptyDir:
      medium: Memory
      sizeLimit: "16Gi"
  - name: torch-compile-cache
    emptyDir: {}
EOF

```

4. 使用 llm-d 的 Helm 圖表部署服務和閘道：

```

cd llm-d/guides/pd-disaggregation/
helmfile apply -e gke_tpu -n $NAMESPACE
kubectl apply -f ./httproute.gke.yaml

```

5. 等待 vLLM 服務啟動觀看解碼和預填 POD 記錄，直到看到「INFO: Application startup complete.」（資訊：應用程式啟動完成）。

```
DECODE_POD=$(kubectl get pods -l llm-d.ai/modelservice-role=decode -o
jsonpath='{.items[0].metadata.name}')

# Get the first Prefill pod name
PREFILL_POD=$(kubectl get pods -l llm-d.ai/modelservice-role=prefill -o
jsonpath='{.items[0].metadata.name}')

echo "Run each of these until vLLM starts successfully and then ctrl-C out"
echo "kubectl logs -f $DECODE_POD -c vllm"
echo "kubectl logs -f $PREFILL_POD -c vllm"
```

步驟 7 · 約 5 分鐘

7. 測試部署作業回應

下方的指令碼會透過 GKE Inference Gateway 測試與服務叢集的連線，然後執行基準化測試。

1. 測試連線並執行基準測試：

```

cat <<EOBF > ./run_benchmark.sh
#!/bin/bash

# Configuration
NAMESPACE="default"
JOB_NAME="qwen3-pd-benchmark"
MODEL_NAME="Qwen/Qwen3-32B"

echo "🔍 Discovering Gateway IP..."
GATEWAY_IP=$(kubectl get gateway -n ${NAMESPACE} -o
jsonpath='{.items[0].status.addresses[0].value}')

if [ -z "$GATEWAY_IP" ]; then
    echo "❌ Error: Could not find Gateway IP. Check 'kubectl get gateway'."
    exit 1
fi

TARGET_URL="http://${GATEWAY_IP}"
echo "✅ Found Gateway at: $TARGET_URL"

echo "🧹 Cleaning up old benchmark jobs..."
kubectl delete job $JOB_NAME --ignore-not-found=true

echo "🚀 Generating and applying Benchmark Job..."
cat <<EOF | kubectl apply -f -
apiVersion: batch/v1
kind: Job
metadata:
name: $JOB_NAME
namespace: $NAMESPACE
spec:
template:
spec:
containers:
- name: llm-benchmark
image: vllm/vllm-openai:latest
command: ["/bin/bash", "-c"]
args:
- |
# 1. Download dataset
if [ ! -f /data/sharegpt.json ]; then
    echo "Downloading ShareGPT dataset..."
    curl -L
"https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered/resolve/main/S
hareGPT_V3_unfiltered_cleaned_split.json" -o /data/sharegpt.json
fi

# 2. Wait for Gateway readiness
echo "Checking connectivity to $MODEL_NAME..."
until curl -s "$TARGET_URL/v1/models" | grep -q "$MODEL_NAME"; do
    echo "Waiting for Gateway backends to sync..."
    sleep 10

```

```
done

# 3. Run Benchmark
vllm bench serve \
  --base-url "$TARGET_URL" \
  --model "$MODEL_NAME" \
  --dataset-name "sharegpt" \
  --dataset-path "/data/sharegpt.json" \
  --request-rate 80.0 \
  --num-prompts 2000 \
  --tokenizer "$MODEL_NAME"
volumeMounts:
- name: dataset-volume
  mountPath: /data
restartPolicy: Never
volumes:
- name: dataset-volume
  emptyDir: {}
EOF

echo "🕒 Job submitted. Follow logs with:"
echo "kubectl logs -f job/$JOB_NAME"
EOBF

chmod a+x ./run_benchmark.sh

./run_benchmark.sh
```

您應該會看到輸出內容，顯示要求處理情形和延遲指標。

步驟 8 · 約 5 分鐘

8. 清除所用資源

如要避免系統持續向您的 Google Cloud 帳戶收費，請刪除本程式碼研究室建立的資源。

請按照下列步驟清理資產：

```

# 1. Delete LeaderWorkerSet and Helm release
kubectl delete leaderworkerset qwen-simple-anywhere-cache --ignore-not-found
helm uninstall lws --namespace lws-system 2>/dev/null
kubectl delete namespace lws-system --ignore-not-found

# 2. Delete GKE Node Pools
# Note: Usually deleting the cluster deletes the node pools,
# but explicit deletion ensures it's gone before the cluster teardown begins.
for i in {1..8}
do
    gcloud container node-pools delete "tpu-v6e-single-$i" \
        --cluster="${CLUSTER_NAME}" \
        --region="${REGION}" \
        --project="${PROJECT_ID}" --quiet

done

# 3. Delete GKE Cluster
gcloud container clusters delete "${CLUSTER_NAME}" \
    --region="${REGION}" \
    --project="${PROJECT_ID}" --quiet

echo "--- Starting IAM and Service Account Cleanup ---"

# 1. Define the full Service Account email for clarity
SA_EMAIL="tpu-reader-sa@${PROJECT_ID}.iam.gserviceaccount.com"

# 2. Remove Storage Bucket IAM Binding
# This removes the 'objectViewer' role from the specific bucket
gcloud storage buckets remove-iam-policy-binding gs://inf-demo-model-storage \
    --member="serviceAccount:${SA_EMAIL}" \
    --role="roles/storage.objectViewer" --quiet

# 3. Remove Workload Identity Binding
# This severs the link between the GKE KSA and the GCP SA
gcloud iam service-accounts remove-iam-policy-binding "${SA_EMAIL}" \
    --role="roles/iam.workloadIdentityUser" \
    --member="serviceAccount:${PROJECT_ID}.svc.id.goog[default/default]" --quiet

# 4. Delete the Service Account
gcloud iam service-accounts delete "${SA_EMAIL}" --project="${PROJECT_ID}" --quiet

echo "IAM cleanup complete!"

echo "--- Starting Network and Firewall Cleanup ---"

# 4. Delete Firewall Rules (Must go before the Network)
gcloud compute firewall-rules delete \
    "${GVNIC_NETWORK_PREFIX}-allow-ssh" \
    "${GVNIC_NETWORK_PREFIX}-allow-icmp" \
    "${GVNIC_NETWORK_PREFIX}-allow-internal" \
    "ray-allow-internal" \

```

```
--project="${PROJECT_ID}" --quiet

# 5. Delete Subnets (Must go before the Network)
gcloud compute networks subnets delete "${GVNIC_NETWORK_PREFIX}-tpu" \
  --region="${REGION}" \
  --project="${PROJECT_ID}" --quiet

gcloud compute networks subnets delete "${GVNIC_NETWORK_PREFIX}-proxy-sub" \
  --region="${REGION}" \
  --project="${PROJECT_ID}" --quiet

gcloud compute networks subnets delete "proxy-only-subnet" \
  --region="${REGION}" \
  --project="${PROJECT_ID}" --quiet

# 6. Finally, delete the VPC Network
gcloud compute networks delete "${GVNIC_NETWORK_PREFIX}-main" \
  --project="${PROJECT_ID}" --quiet

echo "Cleanup complete!"
```

步驟 9 · 約 0 分鐘

9. 恭喜

恭喜！您已成功使用 `llm-d` 和 GKE，在分離式 v6e TPU 上部署 Qwen3-32B。

目前所學內容

- 如何設定自訂網路，以支援高速 TPU 流量。
- 如何在 GKE 上佈建預留 TPU 節點集區。
- 如何部署 `llm-d`，以區隔預填和解碼工作負載。

後續步驟

- [llm-d 說明文件](#)
 - [GKE 說明文件](#)
-